

Locking & Atomic UPDATE

Quick Reference

Spring Boot + JPA + PostgreSQL | Practical decision guide for CineVerse and future systems

The goal of this note is not to memorize annotations. Use it as a decision aid: identify the concurrency problem, choose the smallest mechanism that guarantees the business rule, and then verify it with a concurrency test.

Core idea: Atomic UPDATE, optimistic locking, and pessimistic locking are not interchangeable “lock options”. They solve overlapping but different problems.

For CineVerse seat holding: when the rule is simply AVAILABLE -> HELD only if the seat is still available, an atomic conditional UPDATE is often the simplest fit.

1. The decision in one page

Mechanism	Typical question	Main tool	Best fit
Atomic conditional UPDATE	Can this state transition happen right now?	UPDATE ... WHERE ...	Single-row state transitions
Optimistic locking	Did somebody change this row after I read it?	@Version	Read-modify-write workflows
Pessimistic locking	Must I prevent others from changing the row while I work?	@Lock(PESSIMISTIC_WRITE) / SELECT FOR UPDATE	Critical multi-step transaction

Memory shortcut: atomic UPDATE = condition in the write; optimistic = version conflict; pessimistic = hold a database lock while working.

2. Atomic conditional UPDATE

Use this when the database can decide the whole business transition in one statement. The key pattern is UPDATE ... WHERE current_state_is_expected.

SQL syntax

```
UPDATE show_seat
SET status = 'HELD'
WHERE id = :seatId
      AND status = 'AVAILABLE';
```

The database returns the number of rows changed. In the CineVerse seat-hold case: 1 row = success; 0 rows = the condition was not true when the update executed (commonly another request already claimed the seat).

Spring Data JPA repository syntax

```
@Modifying
@Query("""
    UPDATE ShowSeat s
      SET s.status = com.cineverse.enums.SeatStatus.HELD
    WHERE s.id = :seatId
      AND s.status = com.cineverse.enums.SeatStatus.AVAILABLE
""")
int holdSeat(@Param("seatId") Long seatId);
```

Depending on your JPA setup, enum syntax may differ. The important part is the conditional predicate that protects the state transition.

Service syntax

```
@Transactional
public boolean holdSeat(Long seatId) {
    int updatedRows = showSeatRepository.holdSeat(seatId);
    return updatedRows == 1;
}
```

When to choose it

Choose atomic UPDATE when the operation is a small state transition, the database can express the precondition in the WHERE clause, you mainly need “first successful transition wins”, and you do not need to keep the row locked for several application-level steps.

Typical examples

- Seat: AVAILABLE -> HELD
- Inventory: decrement only when stock > 0
- Job: PENDING -> PROCESSING only if still PENDING
- Idempotency/event processing: mark an event as processed only if it has not already been processed

3. Optimistic locking

Optimistic locking assumes conflicts are possible but does not keep a database lock during the application work. You read a row, someone else may update it, and your write succeeds only if the version is still the one you read.

JPA syntax: @Version

```
@Entity
public class ShowSeat {

    @Id
    private Long id;

    @Enumerated(EnumType.STRING)
```

```

    private SeatStatus status;

    @Version
    private Long version;
}

```

Hibernate conceptually uses a version-aware UPDATE:

```

UPDATE show_seat
SET status = ?, version = version + 1
WHERE id = ?
AND version = ?;

```

If zero rows are updated because the version changed, Hibernate detects the conflict and raises an optimistic-locking exception (commonly `OptimisticLockException` or a Spring-translated equivalent).

When to choose it

Use optimistic locking when you have a read -> decide/modify -> write workflow and want to detect stale data without holding a database lock during the whole business operation.

Good examples

- Editing a movie/show configuration that many admins may update
- Updating a booking record where conflicts are possible but not constant
- Long business workflows where keeping a database row locked would be expensive

Why it is less direct for CineVerse seat hold

For the specific seat-hold operation, you do not need to read the seat, carry a version through application logic, and then detect a stale write. The business rule can be expressed directly as “set HELD only while status = AVAILABLE”. Atomic UPDATE is therefore simpler for this narrow operation.

4. Pessimistic locking

Pessimistic locking asks the database to lock the selected row so competing transactions cannot perform conflicting work on it until the transaction completes.

Spring Data JPA syntax

```

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select s from ShowSeat s where s.id = :seatId")
Optional<ShowSeat> findSeatForUpdate(@Param("seatId") Long seatId);

```

Conceptually, on PostgreSQL this maps to behavior similar to `SELECT ... FOR UPDATE`. Exact SQL is generated by the JPA provider and dialect.

Typical service flow

```

@Transactional
public boolean holdSeat(Long seatId) {
    ShowSeat seat = showSeatRepository.findSeatForUpdate(seatId)
        .orElseThrow(...);

    if (seat.getStatus() != SeatStatus.AVAILABLE) {
        return false;
    }

    seat.setStatus(SeatStatus.HELD);
    return true;
}

```

When to choose it

Use pessimistic locking when you genuinely need to serialize a multi-step transaction around a row and correctness depends on keeping the row protected while several operations happen.

Costs / drawbacks

- Competing transactions may wait for the lock.
- High contention can reduce throughput and increase latency.
- Long transactions hold locks longer and increase contention.
- Deadlocks are possible when multiple rows are locked in inconsistent orders.

For CineVerse seat holding, a pessimistic lock can work, but it is heavier than necessary for a single conditional state transition.

5. Side-by-side comparison

Question	Atomic UPDATE	Optimistic	Pessimistic
Core idea	Make the write conditional	Detect stale version	Lock row while transaction works
Typical syntax	UPDATE ... WHERE status = AVAILABLE	@Version	@Lock(PESSIMISTIC_WRITE)
Conflict result	0 rows updated	Version conflict / exception	Other transaction waits
Best for	Tiny state transitions	Read-modify-write	Multi-step critical transaction
Lock held during app logic?	No	No	Yes, until transaction ends
High contention behavior	Cheap rejection after first success	More conflicts / retries may occur	More waiting / blocking
Complexity	Low	Medium	Higher

6. How to decide in 30 seconds

Question 1: Can I express the correctness rule as one conditional database write?

→ Yes: prefer an atomic UPDATE.

→ No: continue.

Question 2: Is this a read-modify-write workflow where I can tolerate a conflict and retry/fail when the data changed?

→ Yes: consider optimistic locking with @Version.

→ No: continue.

Question 3: Do I need to hold a row lock while performing several dependent operations in one transaction?

→ Yes: consider pessimistic locking.

→ No: reconsider whether a simpler atomic write can express the invariant.

Decision pattern: choose the smallest mechanism that directly guarantees the invariant.

7. CineVerse example

Business invariant: one seat must not be successfully held by two users at the same time.

Preferred narrow operation

```
UPDATE show_seat
SET status = 'HELD'
WHERE id = :seatId
  AND status = 'AVAILABLE';
```

Interpret the affected-row count:

```
updatedRows == 1 -> hold succeeded
updatedRows == 0 -> hold failed; seat was not AVAILABLE
```

This is a good fit because the business rule is local to one row and one transition. There is no need to lock the row while Java performs a long sequence of steps.

When CineVerse might need another mechanism

If a future feature requires loading the seat, inspecting several related rows, making multiple coordinated changes, and keeping the state protected across those changes, a pessimistic lock may become appropriate. If an admin workflow edits a show configuration and stale writes must be detected, optimistic locking may be more appropriate.

8. How to test the choice

For a high-contention seat-hold operation, a unit test alone is not enough. Use an integration/concurrency test that calls the real service/repository/database path.

```
10,000 requests
  ↓
100 worker threads
  ↓
same seatId
  ↓
atomic UPDATE
  ↓
successCount == 1
```

The key assertion is the business invariant: exactly one request succeeds. This gives confidence that the database-level concurrency rule is working under contention.

9. Common mistakes

- Do not read status in Java and then issue a separate unconditional UPDATE if the decision depends on the status. That creates a race window.
- Do not add `@Transactional` and assume it automatically prevents double booking. Transaction boundaries alone do not encode the seat-availability invariant.
- Do not use a pessimistic lock merely because concurrency sounds scary. Locking has a cost.
- Do not use `@Version` just because it is common. Use it when versioned conflict detection fits the workflow.
- Always test the actual concurrency invariant under realistic contention.

10. Final cheat sheet

Need	Choose	Remember
One-row conditional state transition	Atomic UPDATE	Put the precondition in WHERE; inspect affected rows.
Read-modify-write with stale-data detection	Optimistic locking	<code>@Version</code> ; conflict is detected after someone else changed the row.
Multi-step critical transaction needing the row protected	Pessimistic locking	Lock the row; expect waiting and keep transactions short.

CineVerse default for seat acquisition: start with atomic conditional UPDATE when the invariant is a single-row state transition. Move to optimistic or pessimistic locking when the workflow genuinely requires their specific semantics.